



Unidade 2 — Frameworks



Objetivos de Aprendizagem

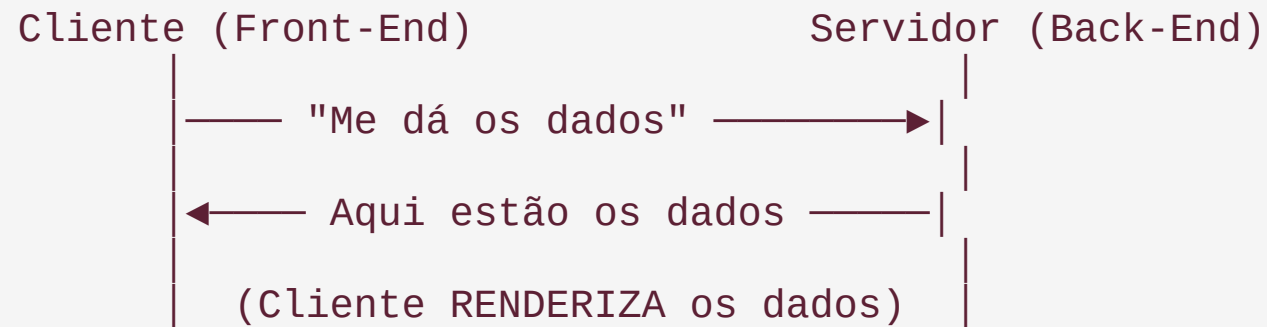
Ao final desta aula, o estudante deverá ser capaz de:

- **Explicar** os paradigmas Pull-based e Push-based;
- **Diferenciar** os dois paradigmas com exemplos concretos;
- **Identificar** frameworks que seguem cada paradigma;
- **Analisar** as vantagens e desvantagens de cada abordagem.

Taxonomia de Bloom: Explicar, Diferenciar, Identificar, Analisar.



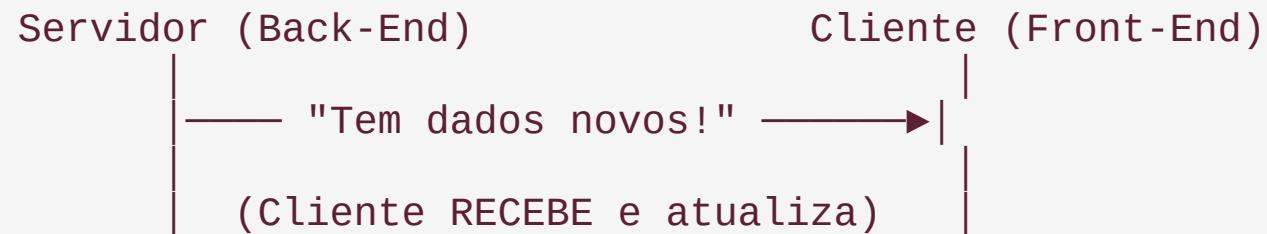
Pull-based (Puxar)



- O **cliente puxa** os dados do servidor;
- O cliente decide **quando** buscar dados;
- O servidor é **reativo** — responde quando solicitado;
- Exemplos: **React**, **Vue.js**, **Angular** (com HTTP).



Push-based (Empurrar)



- O **servidor empurra** dados para o cliente;
- O servidor decide **quando** enviar dados;
- O cliente é **reativo** — atualiza quando recebe;
- Exemplos: **WebSockets, Server-Sent Events (SSE), Firebase.**



Comparativo

Aspecto	Pull-based	Push-based
Quem inicia	Cliente	Servidor
Quando buscar	Quando o cliente precisa	Quando o servidor tem dados novos
Tecnologias	HTTP, fetch, axios	WebSockets, SSE, Firebase
Casos de uso	Listas, formulários, CRUD	Chat, notificações, tempo real
Complexidade	Simples	Mais complexa
Escalabilidade	Fácil	Mais difícil (conexões persistentes)



Exemplo Pull-based (React + fetch)

```
function Tarefas() {
  const [tarefas, setTarefas] = useState([]);

  // Pull: cliente puxa dados quando precisa
  useEffect(() => {
    fetch('/api/tarefas')
      .then(res => res.json())
      .then(data => setTarefas(data));
  }, []);

  return (
    <ul>
      {tarefas.map(t => <li key={t.id}>{t.titulo}</li>)}
    </ul>
  );
}
```



Exemplo Push-based (WebSocket)

```
// Servidor (Node.js + ws)
const WebSocket = require('ws');
const wss = new WebSocket.Server({ port: 8080 });

wss.on('connection', (ws) => {
  // Push: servidor envia dados quando tem novidade
  setInterval(() => {
    ws.send(JSON.stringify({ temperatura: Math.random() * 40 }));
  }, 5000);
});

// Cliente (JavaScript)
const ws = new WebSocket('ws://localhost:8080');
ws.onmessage = (event) => {
  const dados = JSON.parse(event.data);
  console.log('Temperatura:', dados.temperatura);
  // Atualizar interface automaticamente
};
```



Síntese da Aula

Conceitos principais:

1. **Pull-based**: o cliente puxa dados quando precisa (HTTP, fetch);
2. **Push-based**: o servidor envia dados quando tem novidade (WebSockets, SSE);
3. Pull-based é **mais simples** e **mais escalável**;
4. Push-based é **essencial** para aplicações em tempo real;
5. Muitas aplicações **combinam** os dois paradigmas.

Pergunta reflexiva:

Um aplicativo de delivery precisa mostrar a localização do entreguer em tempo real. Qual paradigma é mais adequado? Por quê?

Próxima aula: Como escolher um Framework? — Vamos estudar critérios de avaliação.

